

CNN and Deep Learning Techniques

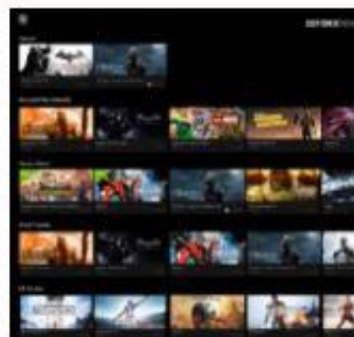
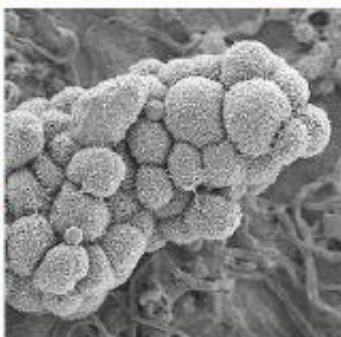


Dr. Satyasai Jagannath Nanda, Senior Member IEEE
Dept. of Electronics & Communication Engineering
Malaviya National Institute of Technology Jaipur
Email : sjnanda.ece@mnit.ac.in

Deep Learning



DEEP LEARNING EVERYWHERE



INTERNET & CLOUD

Image Classification
Speech Recognition
Language Translation
Language Processing
Sentiment Analysis
Recommendation

MEDICINE & BIOLOGY

Cancer Cell Detection
Diabetic Grading
Drug Discovery

MEDIA & ENTERTAINMENT

Video Captioning
Video Search
Real Time Translation

SECURITY & DEFENSE

Face Detection
Video Surveillance
Satellite Imagery

AUTONOMOUS MACHINES

Pedestrian Detection
Lane Tracking
Recognize Traffic Sign

source: developer.nvidia.com/deep-learning-courses

Deep learning



Recommender System

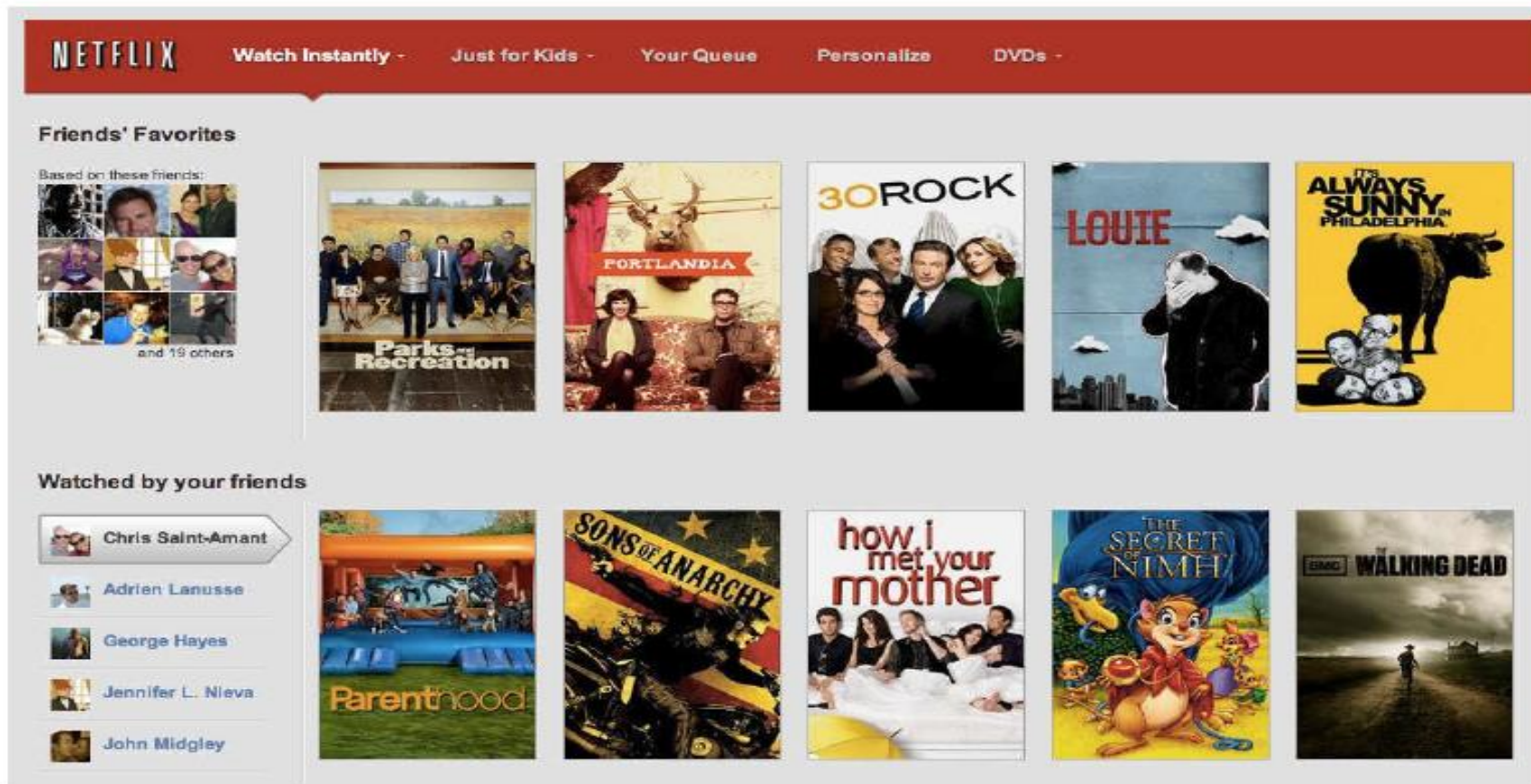


Image courtesy of Netflix

Deep Neural Network



A **Deep neural network** (DNN) is an Artificial **Neural Network** (ANN) with multiple layers between the input and output layers.

Types of Deep Neural Network

- *Convolutional Neural Network*
- *Recurrent Neural Network*
- *AutoEncoders*

What is the need?



Machine Learning

Input: X

Output: Y



Label "motorcycle"

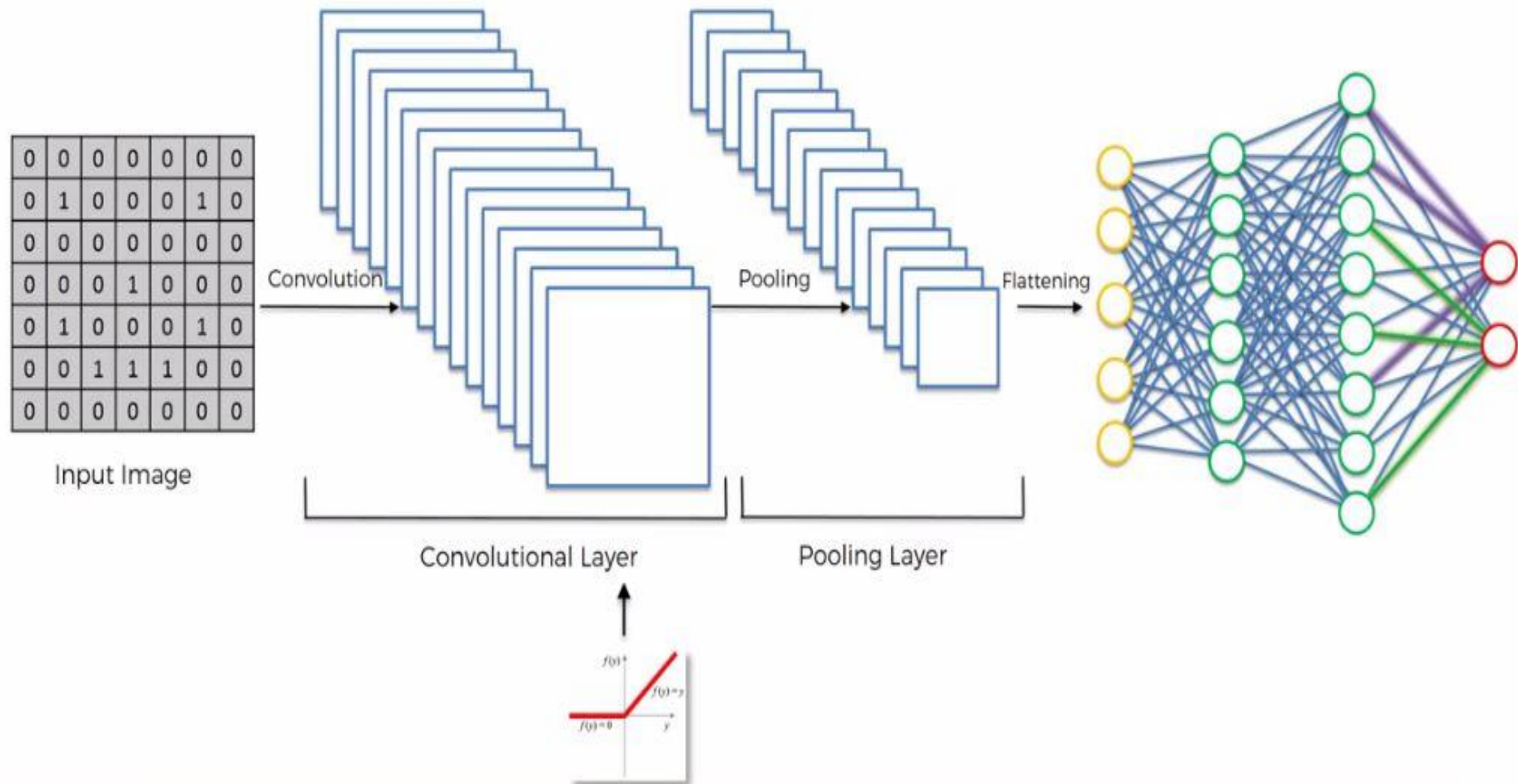
You see this



But the camera sees this:

194	210	201	212	199	213	215	195	176	158	162	209
180	189	190	221	209	205	191	167	147	115	129	163
114	126	140	166	176	155	152	140	170	106	75	88
87	103	115	154	143	142	145	153	173	101	57	57
102	112	108	131	122	138	152	147	128	84	58	66
94	95	79	104	105	124	129	113	107	87	69	67
68	71	69	98	89	92	98	95	89	88	76	67
41	56	65	59	63	45	60	62	58	76	75	65
20	43	69	75	56	41	51	73	55	70	63	44
50	50	57	69	75	75	73	74	53	68	59	37
72	59	53	66	84	92	84	74	57	72	63	42
67	61	58	65	75	78	76	73	59	75	69	50

Convolutional Neural Network

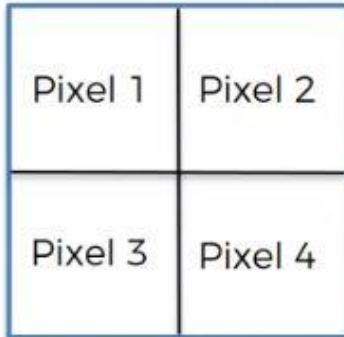


Conceptual Diagram of Convolution Neural Network

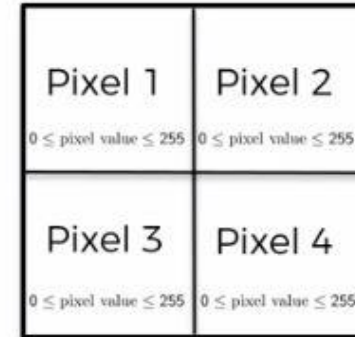
Characteristic of Input Image



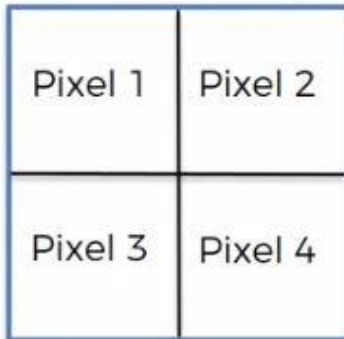
B / W Image 2x2px



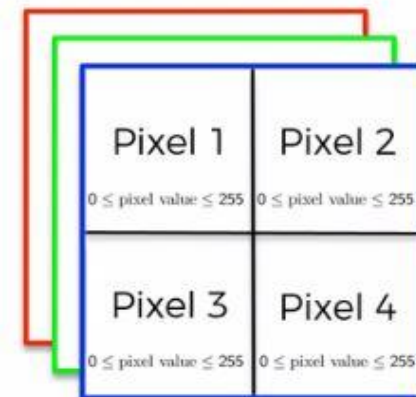
2d array



Colored Image 2x2px



3d array



Feature Detector

Feature detector is the type of filter used to detect features from the image

0	0	0	0	0	0	0
0	1	0	0	0	1	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image



0	0	1
1	0	0
0	1	1

Feature
Detector



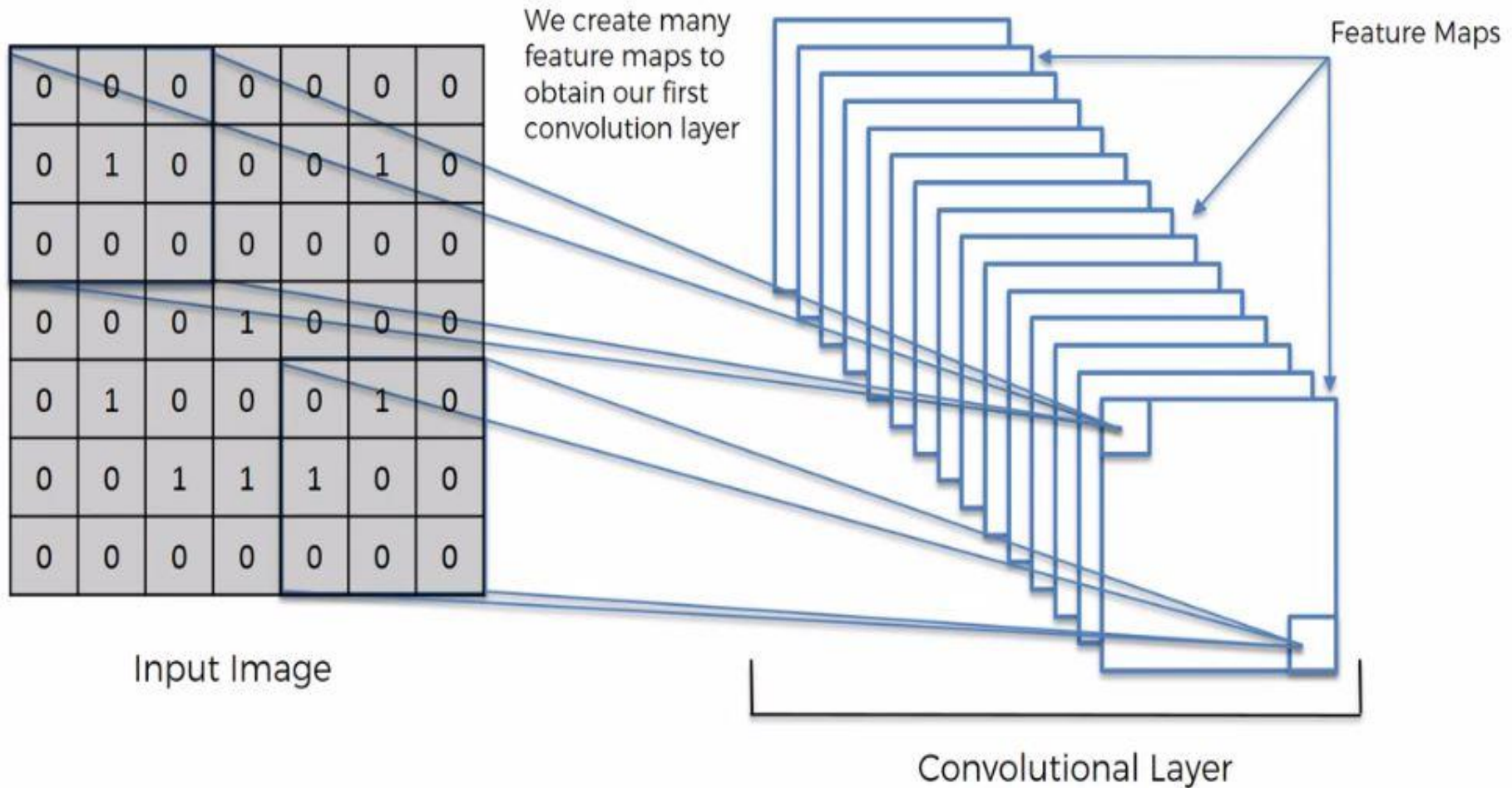
0	1	0	0	0
0	1	1	1	0
1	0	1	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Feature MAP

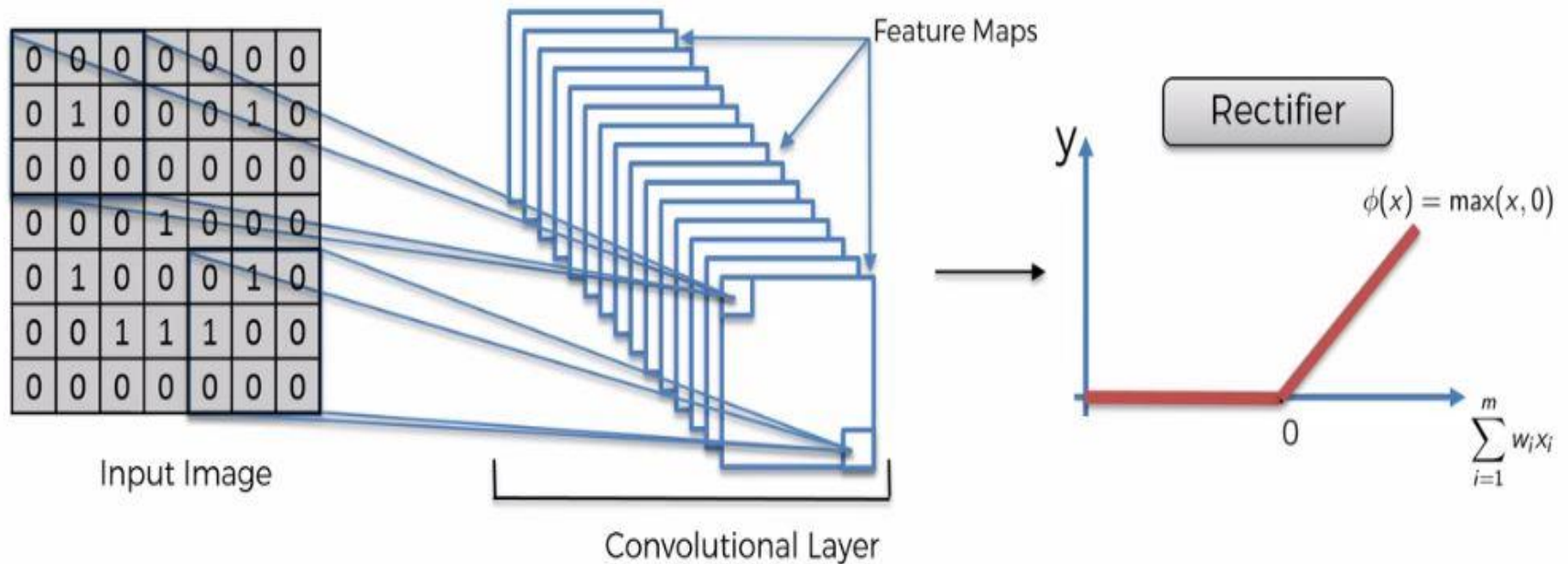


Number of feature MAPs are created to ensure that no Important features are lost



Rectifier Function

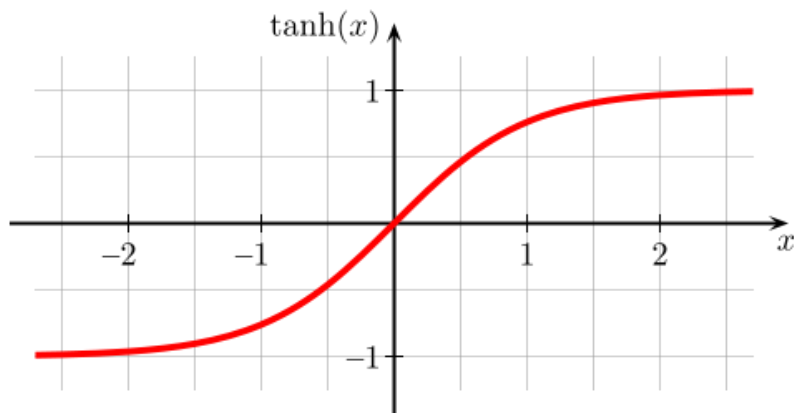
Rectifier Function is Introduced to incorporate the non Linearity in Image



Non-linearity

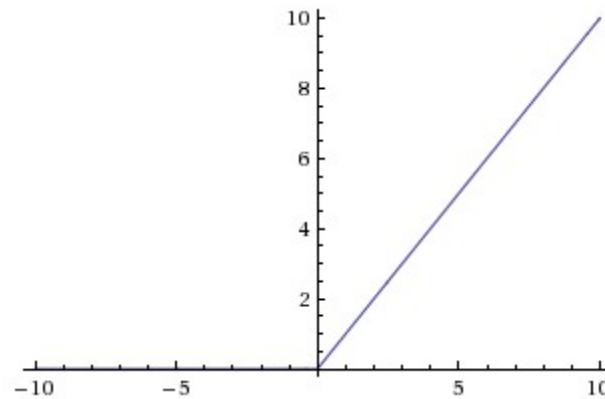


Tanh(x)



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

ReLU- Rectifier Linear Unit



$$f(x) = \max(0, x)$$

Max Pooling

Max Pooling is done to extract the important features and Compress the image

Different type of Pooling is –

- Max Pool
- Mean Pool
- Sum Pool

0	1	0	0	0
0	1	1	1	0
1	0	1	2	1
1	4	2	1	0
0	0	1	2	1

Feature Map

Max Pooling



1	1	0
4		

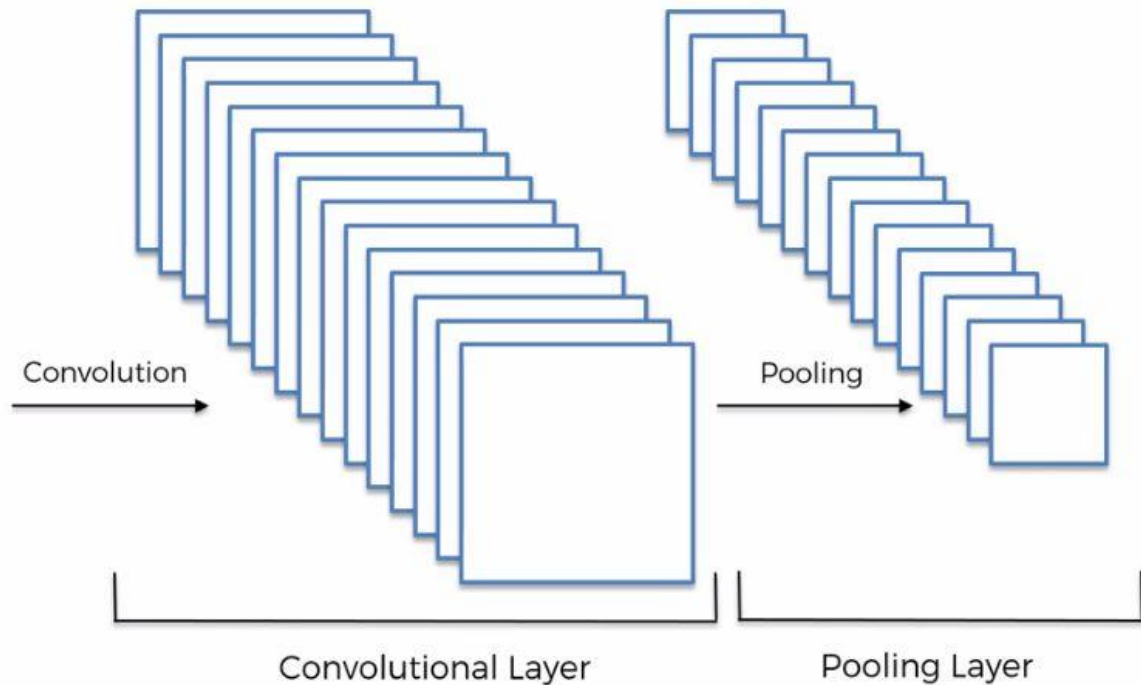
Pooled Feature Map

Pooling Layers



0	0	0	0	0	0	0
0	1	0	0	0	1	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image



Flattening



1	1	0
4	2	1
0	2	1

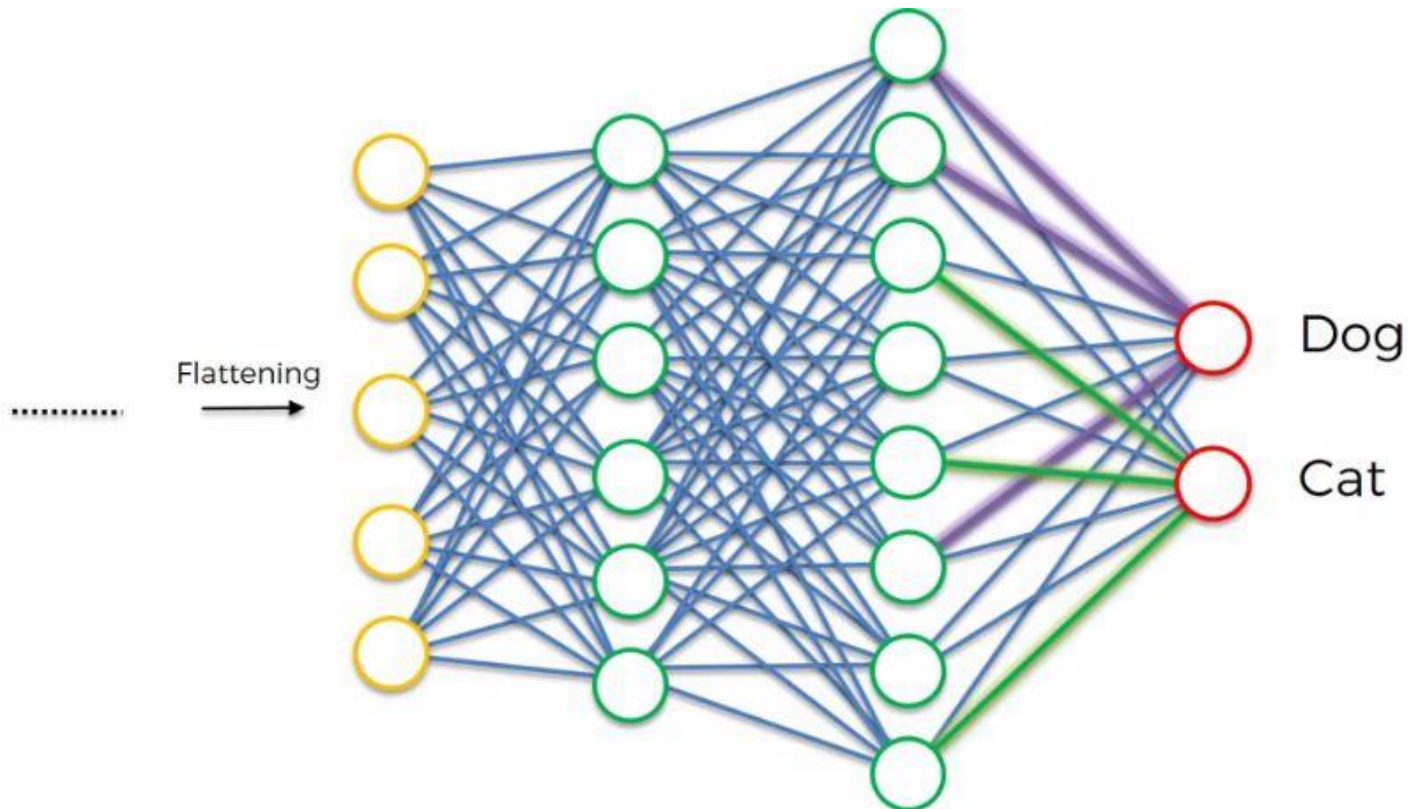
Pooled Feature Map

Flattening



1
1
0
4
2
1
0
2
1

Artificial Neural Network



Bench-Mark Dataset



CIFAR 10 and Convolutional Neural Network

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



CIFAR 10 dataset:

50,000 training images

10,000 testing images

10 categories (classes)

Accuracies from different methods:

Human: ~94%

Whitening K-mean: 80%

.....

Deep CNN: 95.5%

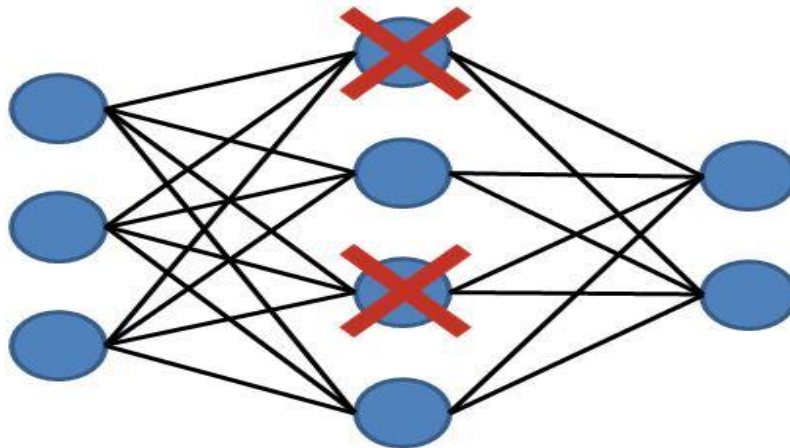
<http://karpathy.github.io/2011/04/27/manually-classifying-cifar10/>

<https://www.kaggle.com/c/cifar-10/leaderboard>

Hyper Parameters in CNN

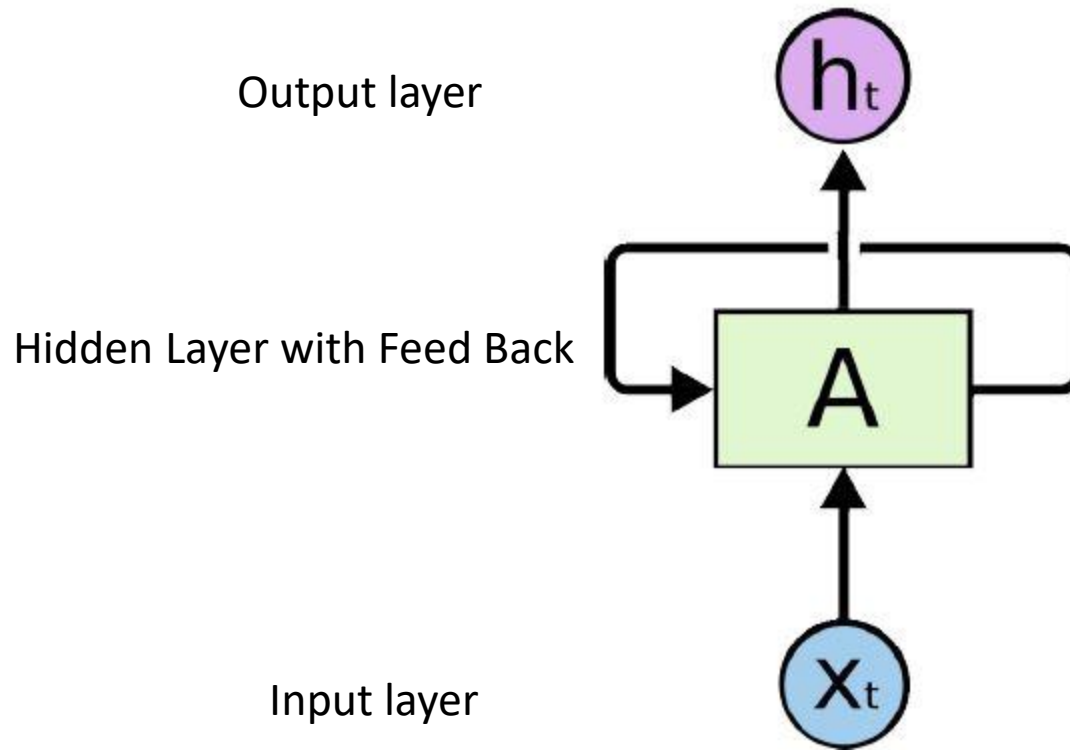
There are number of Hyper-Parameters in The Deep Neural Network

- 1) **Number of Hidden Layers and units**
- 2) **Dropout**- Dropout is regularization technique to avoid over fitting (increase the validation accuracy) thus increasing the generalizing power



- 3) **Network Weight Initialization**- uniform distribution
- 4) **Activation function**- used to **introduce nonlinearity** to models, which allows deep learning models to learn nonlinear prediction boundaries.
 - a) **Sigmoid** is used in the output layer while making **binary predictions**.
 - b) **Softmax** is used in the output layer while making **multi-class predictions**.

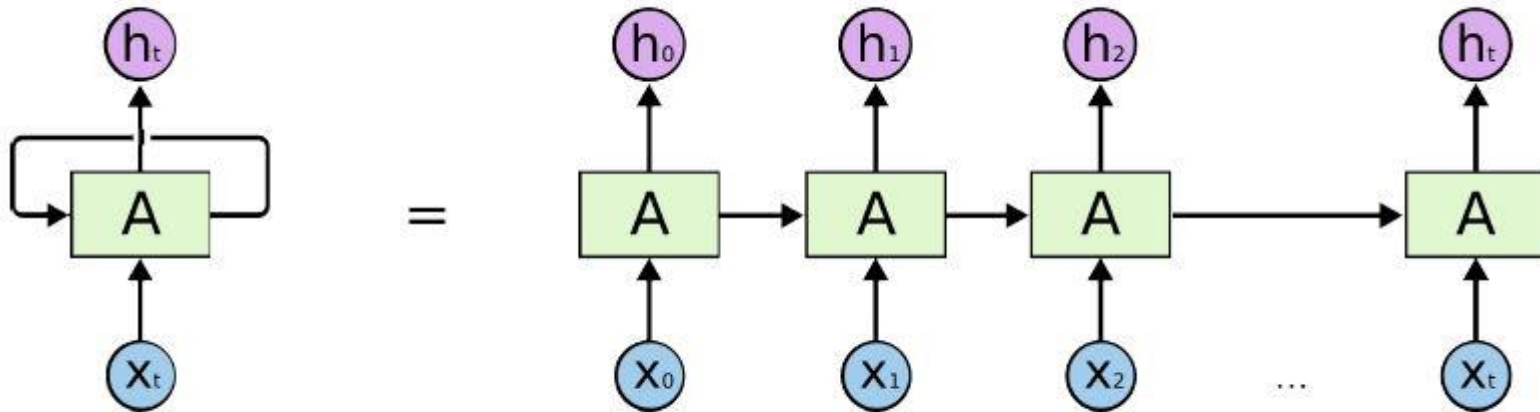
Recurrent Neural Network



Unfolding RNN



A recurrent neural network can be thought of as multiple copies of the same network, each passing a message to a successor

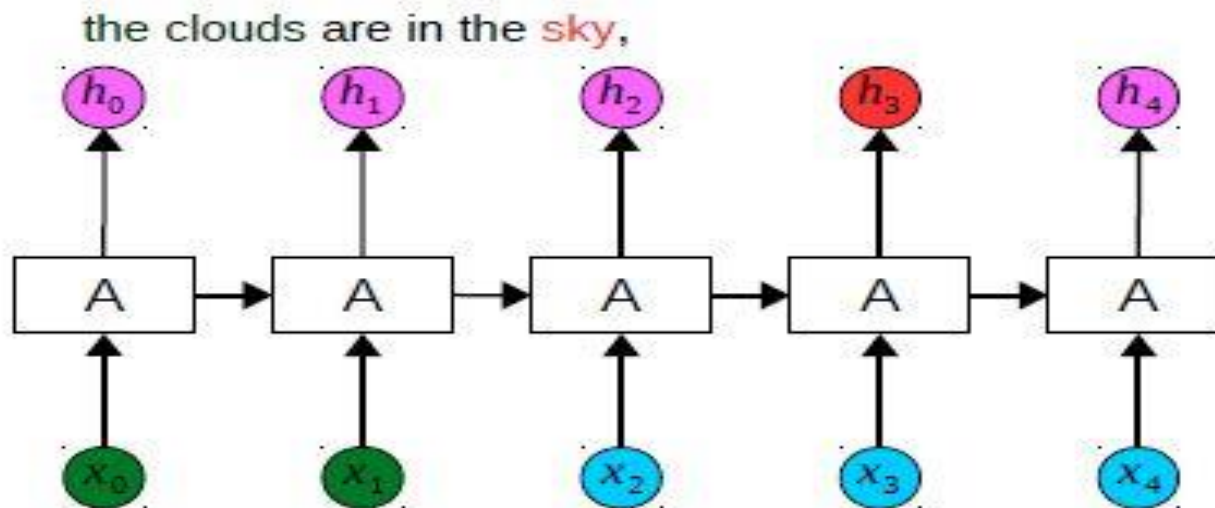


Understanding RNN



- RNN Short Term Dependencies

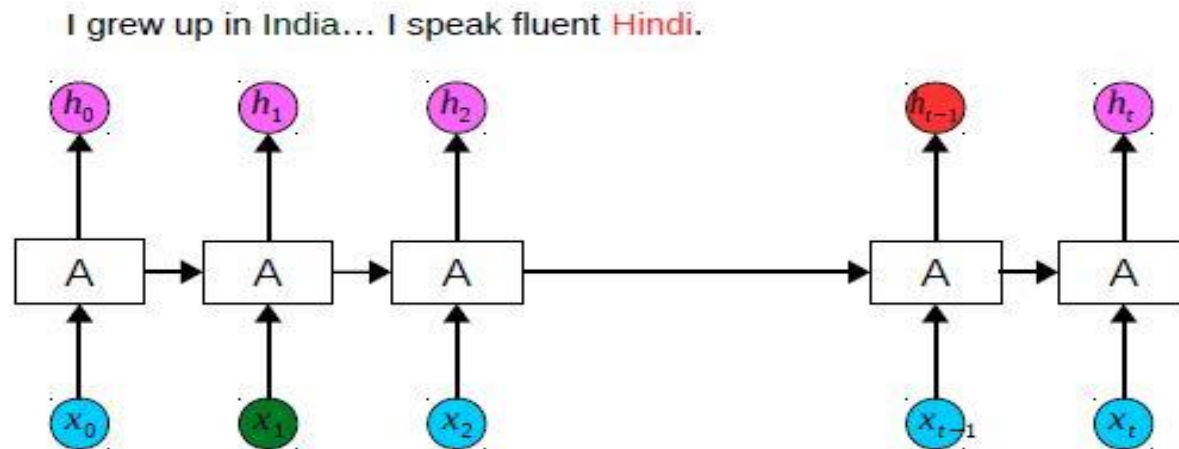
Language model trying to predict the next word based on the previous ones



Understanding RNN



- RNN Short Term Dependencies
- Language model trying to predict the next word based on the previous ones



Why do we need RNNs?



The **limitations** of the Neural network (CNNs)

- Rely on the assumption of independence among the (training and test) examples.
 - After each data point is processed, the entire state of the network is lost

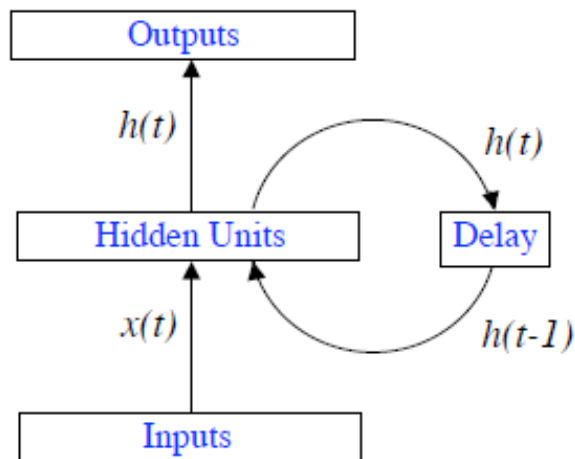
We need to model the data with temporal or sequential structures and varying length of inputs and outputs

- Frames from video
- Snippets of audio
- Words pulled from sentences

What are RNNs?



Recurrent neural networks (RNNs) are connectionist models with the ability to selectively pass information across sequence steps, while processing sequential data one element at a time.



The simplest form of **fully recurrent neural network** is an MLP with the previous set of hidden unit activations feeding back into the network along with the inputs

Allow a 'memory' of previous inputs to persist in the network's internal state, and thereby influence the network output

$$h(t) = f_H(W_{IH}x(t) + W_{HH}h(t-1))$$

$$y(t) = f_O(W_{HO}h(t))$$

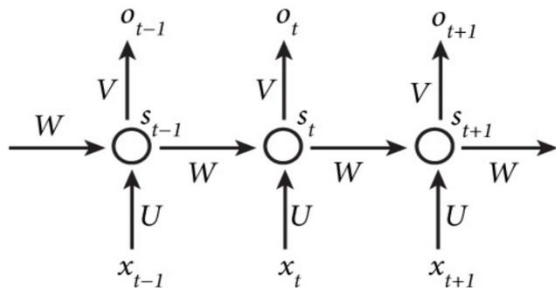
f_H and f_O are the activation function for hidden and output unit; W_{IH} , W_{HH} , and W_{HO} are connection weight matrices which are learnt by training

What are RNNs?



- Training RNNs (determine the parameters)

Back Propagation Through Time (BPTT) is often used to learn the RNN
BPTT is an extension of the back-propagation (BP)

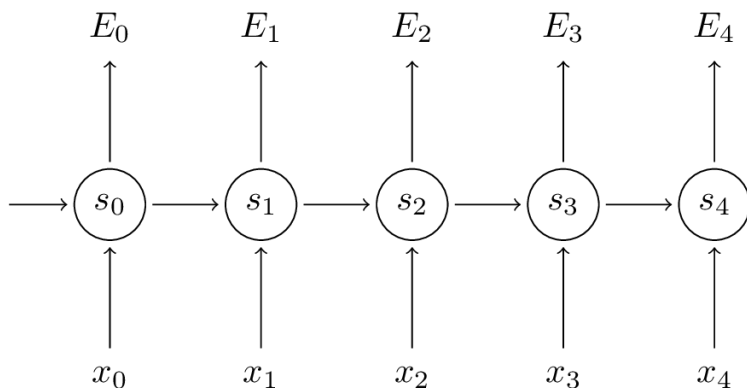


- The output of this RNN is $\hat{y}_t$

$$s_t = \tanh(Ux_t + Ws_{t-1})$$

$$\hat{y}_t = \text{softmax}(Vs_t)$$

- The loss/error function of this network is



$$E_t(y_t, \hat{y}_t) = -y_t \log \hat{y}_t \rightarrow$$

The error at each time step

$$E(y, \hat{y}) = \sum_t E_t(y_t, \hat{y}_t) \rightarrow$$

the total loss is the sum of the errors at each time step

What are RNNs?



- Training RNNs (determine the parameters)
 - ✓ The gradients of the error with respect to our parameters
*Just like we sum up the errors, we also **sum up the gradients at each time step for one training example**. For parameter W , the gradient is*

$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}$$

- ✓ The gradient at each time step

we use time 3 as an example

$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial W} \longrightarrow \text{Chain Rule}$$

$$s_3 = \tanh(Ux_1 + Ws_2) \longrightarrow \begin{array}{l} s_3 \text{ depends on } W \text{ and } s_1, \text{ we cannot simply} \\ \text{treat } s_2 \text{ a constant} \end{array}$$

$$\frac{\partial E_3}{\partial W} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial W} \longrightarrow \text{Apply Chain Rule again on } s_k$$

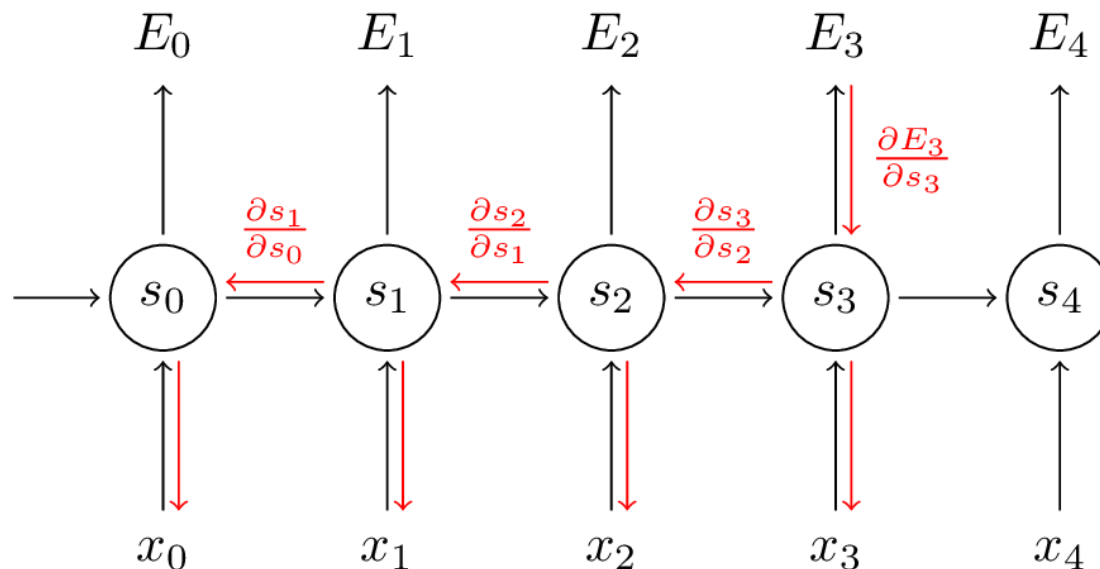
What are RNNs?



- Training RNNs (determine the parameters)

$$\frac{\partial E_3}{\partial W} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial W}$$

Because W is used in every step up to the output we care about, we need to back-propagate gradients from $t = 3$ through the network all the way to $t = 0$



What are RNNs?

- The vanishing gradient problem

To understand why, let's take a closer look at the gradient we calculated above:

$$\frac{\partial E_3}{\partial W} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial W} \longrightarrow \frac{\partial E_3}{\partial W} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \underbrace{\prod_{j=k+1}^3 \frac{\partial s_j}{\partial s_{j-1}}}_{\text{vanishing}} \frac{\partial s_k}{\partial W}$$

Because the layers and time steps of deep neural networks relate to each other through multiplication, derivatives are susceptible to vanishing

Gradient contributions from “far away” steps become zero, and the state at those steps doesn't contribute to what you are learning: You end up not learning long-range dependencies.

What are RNNs?



● How to solve the vanishing gradient problem?

- ❑ Proper initialization of the W matrix can reduce the effect of vanishing gradients
- ❑ Use ReLU instead of tanh or sigmoid activation function
ReLU derivative is a constant of either 0 or 1, so it isn't likely to suffer from vanishing gradients
- ❑ Use Long Short-Term Memory or Gated Recurrent unit architectures
LSTM will be introduced later

Vanishing Gradient Problem



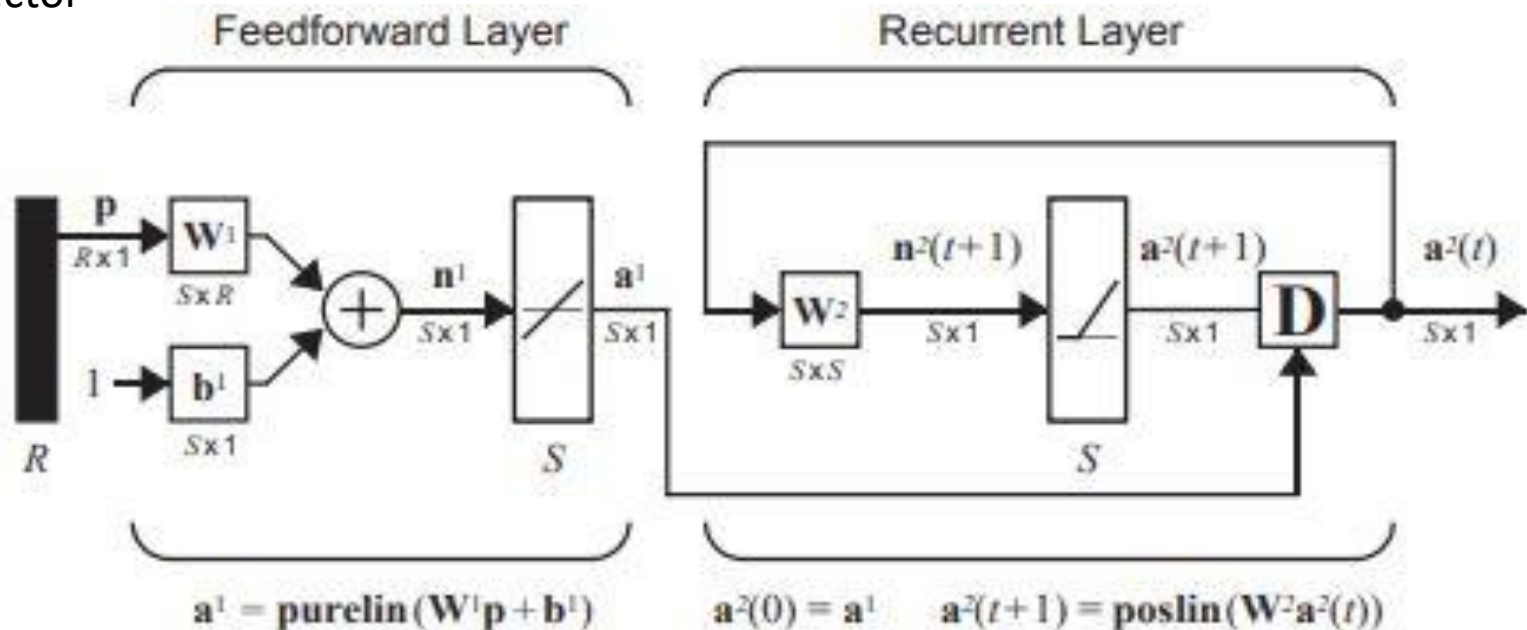
$$\frac{\partial E_3}{\partial \mathbf{W}} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial \mathbf{W}}$$

$$\frac{\partial E_3}{\partial \mathbf{W}} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \left(\prod_{j=k+1}^3 \frac{\partial s_j}{\partial s_{j-1}} \right) \frac{\partial s_k}{\partial \mathbf{W}}$$

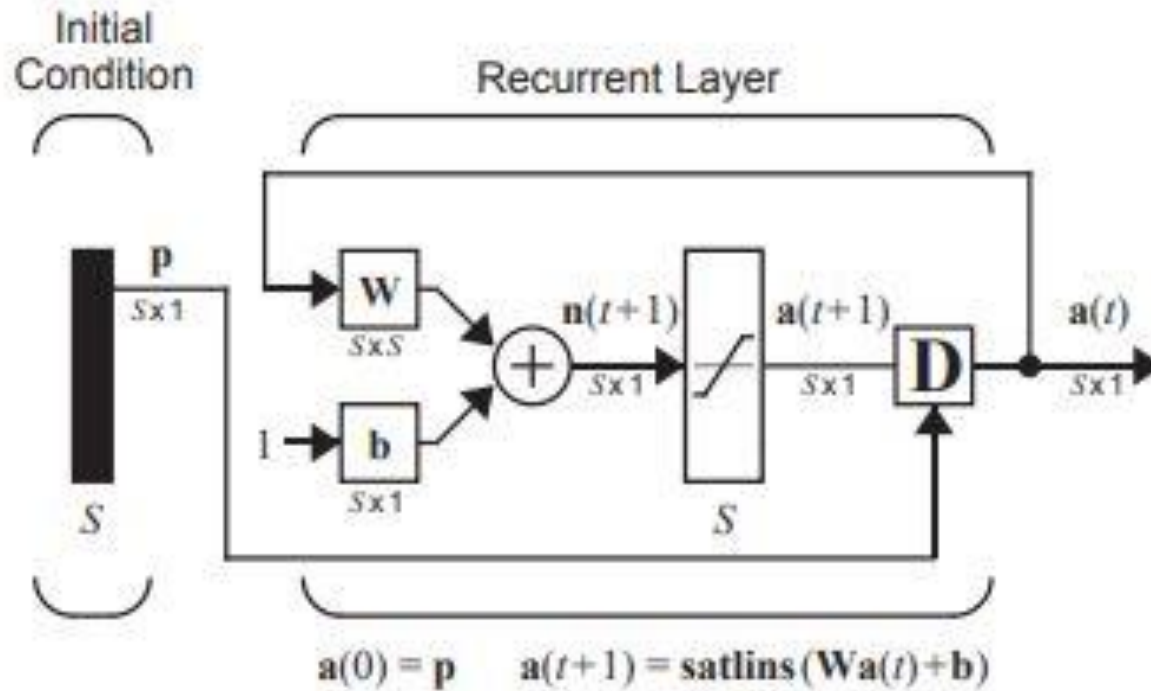
- Derivative of a vector w.r.t a vector is a matrix called jacobian
- 2-norm of the above Jacobian matrix has an upper bound of 1
- **tanh** maps all values into a range between -1 and 1, and the **derivative is bounded by 1**
- With multiple matrix multiplications, gradient values **shrink exponentially**
- Gradient contributions from “far away” steps become zero
- Depending on activation functions and network parameters, gradients could **explode** instead of vanishing

Hamming Network

- This network is called the **Hamming network** because the neuron in the feedforward layer with the largest output will correspond to the prototype pattern that is closest in Hamming distance to the input pattern
- Used to solve binary pattern recognition problems
- **feed forward**- The feedforward layer performs a correlation, or inner product, between each of the prototype patterns and the input pattern
- **Recurrent (feedback) layers**- neurons in this layer are initialized with the outputs of the feedforward layer, which indicate the correlation between the prototype patterns and the input vector



Hopfield Network



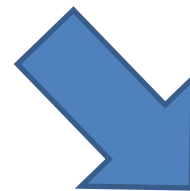
RNN Application



- What can RNNs can do?



Machine Translation



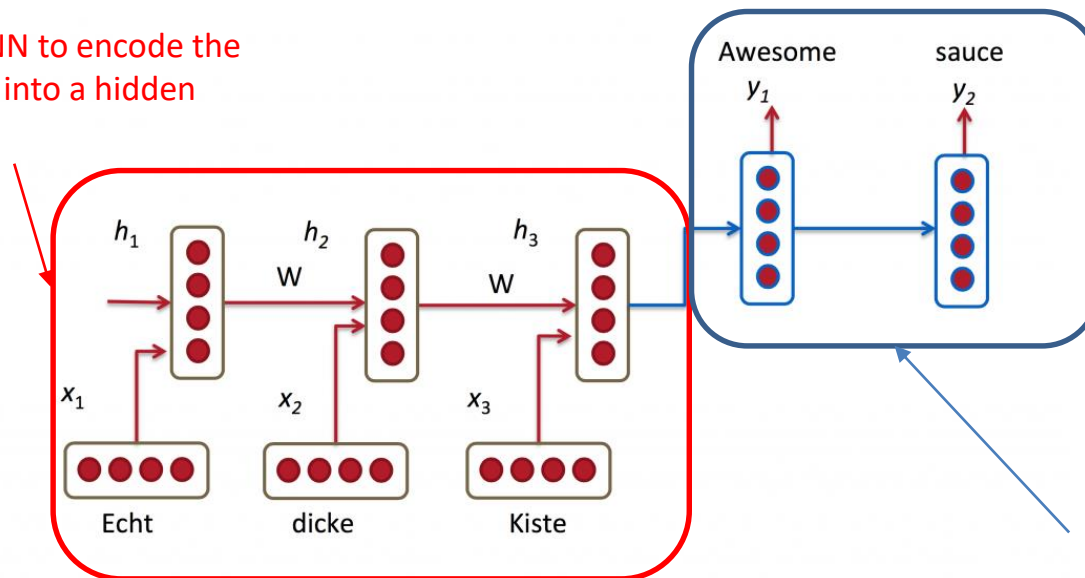
Visual Question Answering

Machine Translation- Google translator



In machine translation, the input is a sequence of words in source language, and the output is a sequence of words in target language.

Encoder: An RNN to encode the input sentence into a hidden state (feature)



Encoder-decoder architecture for machine translation

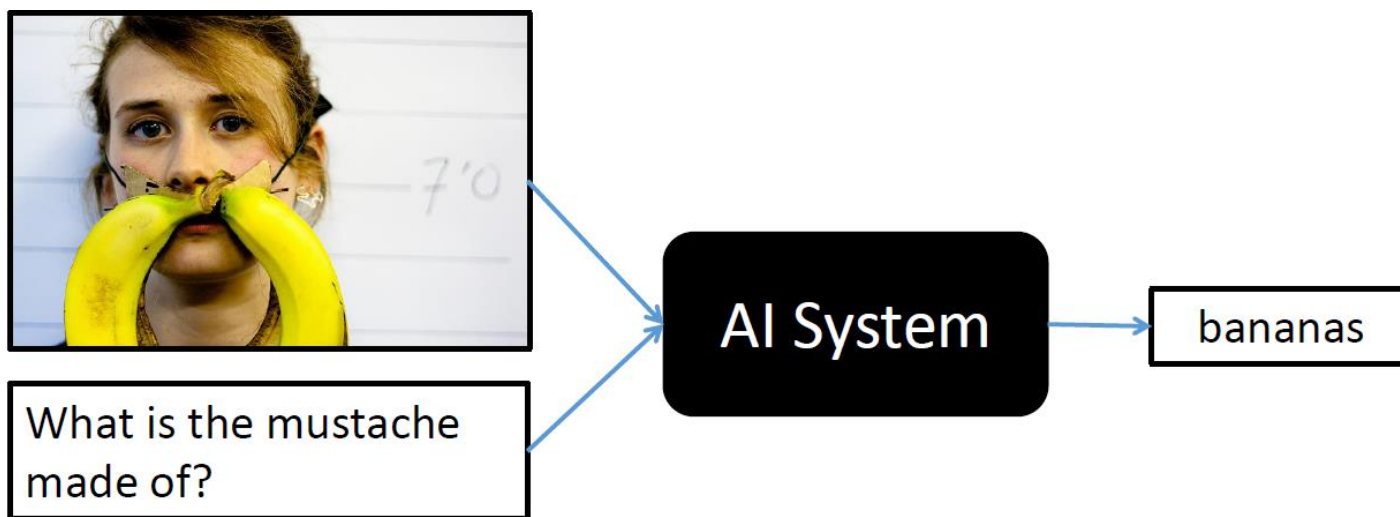
Decoder: An RNN with the hidden state of the sentence in source language as the input and output the translated sentence

Visual Question Answering (VQA)



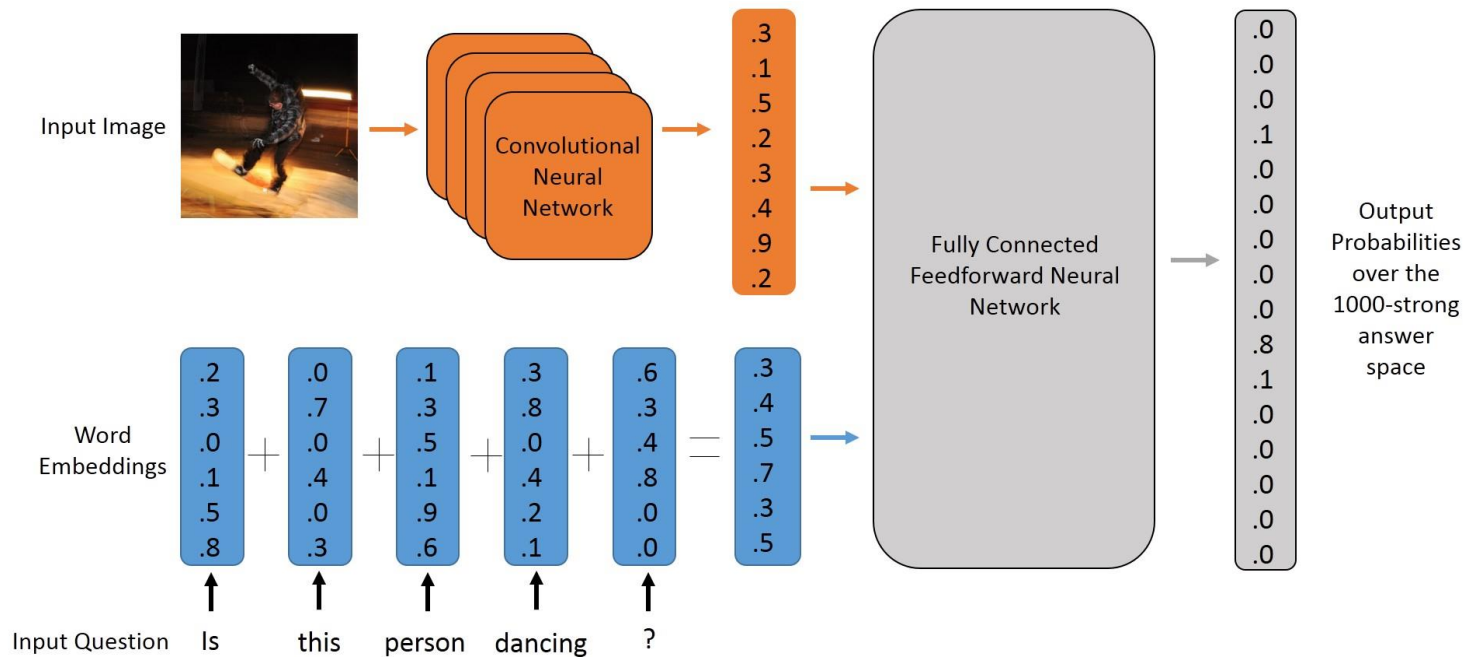
Demo Website

VQA: Given an image and a natural language question about the image, the task is to provide an accurate natural language answer



Picture from (Antol et al., 2015)

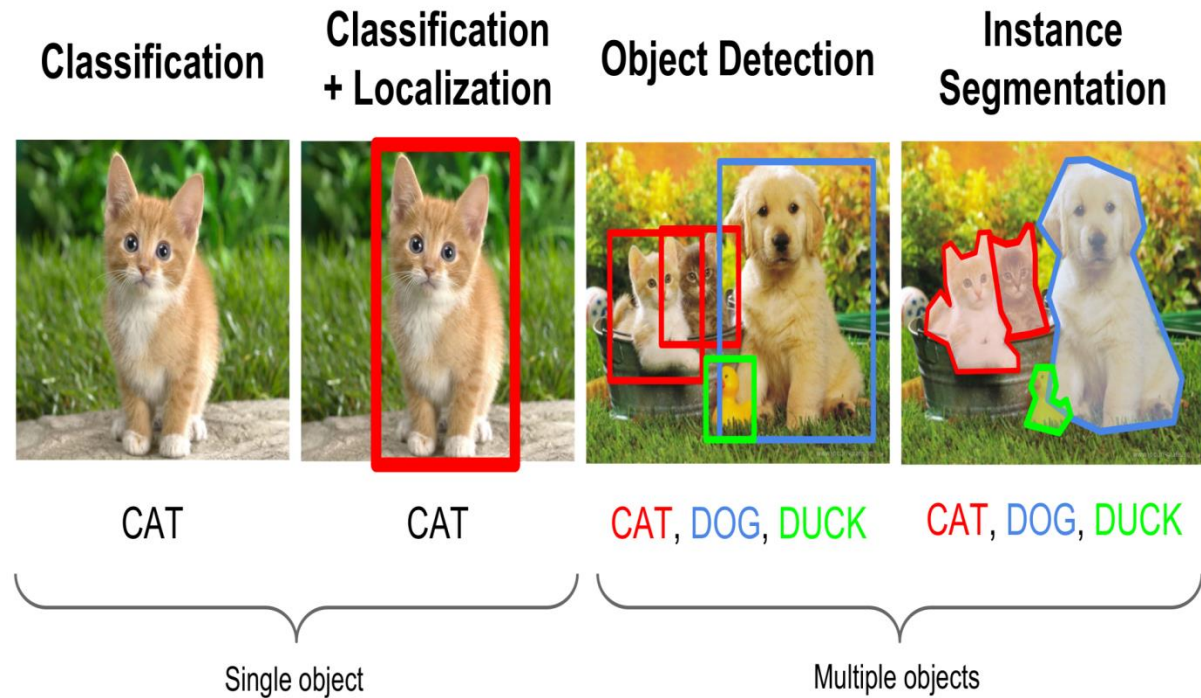
Visual Question Answering



The output is to be conditioned on both image and textual inputs. A CNN is used to encode the image and a RNN is implemented to encode the sentence.

Supervised Learning

- Supervised Learning
- Data: (X,Y)
- Goal: Learn a Mapping Function f where:
- $f(X) = Y$



Supervised Learning



- Examples:
Classification.

Decision
Trees

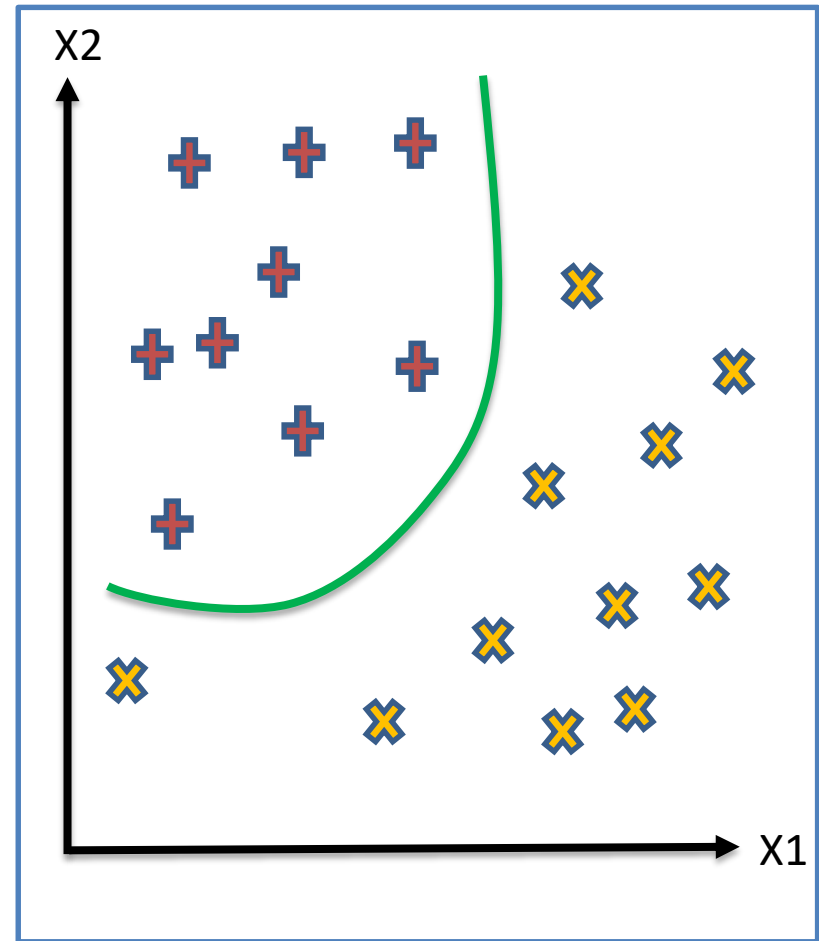
Naïve Bayes

KNN

SVM

Perceptron

Multi Layer
Perceptron



Classification

Unsupervised Learning



Examples: Clustering, **Compression, Feature & Representation learning, Dimensionality reduction**, Generative models ,etc.

References



1. <https://arxiv.org/pdf/1206.5538.pdf>
2. <http://www.deeplearningbook.org/contents/autoencoders.html>
3. <http://deeplearning.net/tutorial/dA.html>
4. <http://ufldl.stanford.edu/tutorial/unsupervised/Autoencoders/>
5. http://ufldl.stanford.edu/wiki/index.php/Stacked_Autoencoders
6. <http://www.jmlr.org/papers/volume11/vincent10a/vincent10a.pdf>
7. <https://codeburst.io/deep-learning-types-and-autoencoders-a40ee6754663>
8. www.superdatascience.com
9. Antol, S., Agrawal, A., Lu, J., Mitchell, M., Batra, D., Lawrence Zitnick, C., & Parikh, D. (2015). VQA: Visual question answering. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 2425-2433).
10. Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504-507.
11. Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *Signal Processing, IEEE Transactions on*, 45(11), 2673-2681.
12. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems* (pp. 1097-1105).
13. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
14. Mnih, Volodymyr, et al. "Playing atari with deep reinforcement learning." *arXiv preprint arXiv:1312.5602* (2013).
15. Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." *Nature* 518.7540 (2015): 529-533.
16. Silver, David, et al. "Mastering the game of Go with deep neural networks and tree search." *Nature* 529.7587 (2016): 484-489.
17. Silver, D. (2015). Deep Reinforcement Learning [Powerpoint slides]. Retrieve from <http://www.iclr.cc/lib/exe/fetch.php?media=iclr2015:silver-iclr2015.pdf>
18. Lecun, Y., & Ranzato, M. (2013). Deep Learning Tutorial [Powerpoint slides]. Retrieved from <http://www.cs.nyu.edu/~yann/talks/lecun-ranzato-icml2013.pdf>



THANK YOU